Complemento de la propuesta de PulsoCheck

Documento complementario de la propuesta inicial | PulsoCheck

Estado de este documento: diseño y planificación. Las funciones descritas aquí se implementarán por etapas; no significa que ya estén terminadas.

1. Arquitectura

PulsoCheck se organizará como una aplicación Android con dos módulos y capas internas. El módulo `wear` contendrá la experiencia del reloj y el módulo `mobile` la experiencia del teléfono. Cada módulo tendrá su propia interfaz y almacenamiento local, y ambos compartirán los modelos de datos que acordemos cuando la sincronización esté diseñada.

Dentro de cada módulo se separarán estas responsabilidades:

1. **Presentación:** pantallas de Compose y estado que se muestra a la persona. La pantalla no accederá directamente al sensor ni a la base de datos.
2. **Dominio:** reglas del proyecto, como validar lecturas y decidir si forman un episodio. Se mantendrán independientes de Android para que puedan entenderse y verificarse con datos de ejemplo.
3. **Datos:** repositorios que coordinan Room, Health Services y Wearable Data Layer. Esta capa convierte la información externa a modelos propios de PulsoCheck.

La presentación utilizará **MVVM**: una pantalla Compose observa el estado de un `ViewModel` y le comunica acciones del usuario. Por ejemplo, el botón de seguimiento llama una función del `ViewModel`; no modifica el sensor directamente.

Fundamentación

El proyecto tiene dos interfaces y varias fuentes de datos (sensor, base local y conexión entre dispositivos). Separarlas reduce el acoplamiento: un cambio en la interfaz no obliga a reescribir las reglas, y una sustitución de almacenamiento no debe cambiar las pantallas. Mantener los módulos `wear` y `mobile` separados también permite desarrollar e instalar cada aplicación de acuerdo con las capacidades de su dispositivo.

La arquitectura será deliberadamente sencilla. No se crearán capas o abstracciones sin una necesidad concreta; cada pieza se añadirá con una explicación y un cambio pequeño que podamos revisar.

2. Patrones de diseño

Los patrones se aplicarán cuando aparezca la necesidad correspondiente:

Patrón	Aplicación prevista	Motivo
MVVM	`ViewModel` entre Compose y las acciones/estado de pantalla.	Mantener la interfaz enfocada en mostrar datos y recibir interacciones.
Repository	Repositorios para lecturas, episodios, respuestas y sincronización.	Evitar que las reglas y pantallas dependan directamente de Room o de un servicio de Google.
Observer / flujo reactivo	`Flow` de Room y estado observable en el `ViewModel`.	Actualizar el historial cuando se guarden datos, sin que la pantalla tenga que consultar repetidamente.

Patrón	Aplicación prevista	Motivo
Adapter	Convertidores entre eventos de Health Services, mensajes de Data Layer y modelos de PulsoCheck.	Aislar formatos externos y mantener un modelo interno estable.
State	Estados como `Detenido`, `Siguiendo`, `Pendiente de sincronizar` y `Sincronizado`.	Hacer explícito qué está ocurriendo y evitar combinaciones ambiguas de valores booleanos.

No se afirmará que un patrón está implementado hasta que exista código que lo aplique. En particular, `TrackingViewModel` actualmente solo mantiene el estado temporal de la demostración del botón; aún no gestiona el sensor, persistencia ni sincronización.

3. Funciones previstas

Los nombres de las funciones siguientes son ejemplos de organización para el código futuro. Se pueden ajustar al implementarlas y explicarlas.

Área	Funciones previstas	Responsabilidad
Seguimiento	`iniciarSeguimiento()`, `detenerSeguimiento()`	Comprobar permisos/capacidades e iniciar o detener la captura disponible.
Lecturas	`guardarLectura(lectura)`, `validarLectura(lectura)`	Validar hora y valor, y conservar la lectura con origen e identificador.
Episodios	`evaluarLecturas(lecturas)`, `cerrarEpisodio(id)`	Aplicar la regla configurable a lecturas consecutivas; no emitir un diagnóstico.
Respuesta	`guardarRespuesta(episodioId, opcion, fechaHora)`	Asociar la carita elegida con el episodio y conservar la hora de respuesta.
Sincronización	`crearLotePendiente()`, `recibirLote(lote)`, `confirmarRecepcion(ids)`	Transferir registros identificables, evitar duplicados y conservar pendientes hasta confirmación.
Historial	`observarHistorial(inicio, fin)`, `calcularResumen(lecturas)`	Mostrar datos de los cinco días locales y resumir solo las lecturas disponibles.
Avisos	`notificarEpisodio(episodioId)`	Presentar una notificación local para un episodio reciente, respetando permisos y ajustes del sistema.

La regla inicial descrita en la propuesta considera tres lecturas válidas consecutivas por encima del límite para abrir un episodio y dos lecturas consecutivas dentro del límite para cerrarlo. Se reiniciará el conteo cuando falten datos o exista una separación excesiva. Estos parámetros son de software y no criterios clínicos; se harán configurables si el alcance y la asesora lo permiten.

4. Intercambio de datos entre reloj y teléfono

La aplicación utilizará **Wearable Data Layer** para comunicarse entre los módulos instalados. No implementará su propio emparejamiento Bluetooth: la vinculación reloj-teléfono queda a cargo de Google Pixel Watch y Android. Data Layer puede aprovechar Bluetooth y, cuando esté disponible, otra conexión de red.

- `DataClient`**:** previsto para datos persistentes o bloques de mediciones, episodios y respuestas.
- `MessageClient`**:** reservado para acciones breves que necesiten comunicación inmediata cuando haya conexión; no será el único medio para conservar datos.
- **Identificador único:** cada lectura, episodio y respuesta tendrá un identificador estable. Al recibir un registro repetido, el almacenamiento lo reconocerá para no duplicarlo.

- **Confirmación:** el teléfono confirmará los identificadores recibidos. El reloj mantendrá los elementos pendientes hasta la confirmación; si no hay conexión, los enviará más tarde.
- **Datos atrasados:** se agregarán al historial, pero no producirán una notificación que parezca indicar un episodio actual.

La operación de seguimiento y la respuesta en el reloj no dependerán de que el teléfono esté conectado.

5. Notificaciones, cámara y sensores

Notificaciones y vibración

El reloj mostrará una notificación local para un episodio reciente y solicitará la respuesta desde la aplicación del propio reloj. La notificación no abrirá forzosamente la pantalla. La vibración dependerá de las capacidades, permisos, modo No molestar y ajustes de notificación del dispositivo. El teléfono podrá mostrar un aviso informativo al recibir un episodio reciente; abrirá su detalle, no el formulario de respuesta.

Cámara

La cámara **no se utilizará** en la primera versión. La propuesta no define una función que la necesite: la respuesta se elige con controles táctiles y el pulso procede del sensor del reloj. Excluir cámara evita pedir un permiso sin justificación y reduce la recolección de datos personales.

Sensor de frecuencia cardiaca

En el reloj se consultará la capacidad `HEART_RATE_BPM` de **Health Services** y se solicitarán los permisos requeridos. `MeasureClient` se considera para mediciones breves mientras la aplicación está abierta. Para lecturas en segundo plano se evaluará `PassiveMonitoringClient` y su receptor, solo si el dispositivo y los permisos lo admiten.

Las lecturas pasivas pueden llegar agrupadas o con retraso. La aplicación mostrará su hora real y no prometerá mediciones exactas cada minuto ni vigilancia continua. La validación del retraso se realizará en el Pixel Watch disponible. No se provocarán elevaciones de pulso para probar el sistema.

6. Base de datos

Room sobre SQLite almacenará los datos localmente en cada dispositivo, para que el reloj pueda registrar un episodio y una respuesta sin conexión. Las entidades previstas son:

- **Lectura:** identificador, valor en lpm, fecha/hora, origen y estado de sincronización.
- **Episodio:** identificador, inicio, fin, lecturas asociadas y estado de respuesta.
- **Respuesta:** identificador del episodio, opción elegida y fecha/hora de guardado.
- **Configuración:** parámetros de prueba y valores que deban compartirse con el reloj.

Los DAO de Room contendrán operaciones de consulta/guardado; el repositorio coordinará esas operaciones con reglas y sincronización. El historial se calculará con datos existentes. Un día sin lecturas se marcará **Sin datos**; no se inventarán valores ni se unirán huecos en la gráfica. Se ofrecerá la eliminación de registros.

7. APIs internas y externas

En este proyecto, “API” puede referirse a funciones de Android/Google usadas dentro de la aplicación; no implica que exista un servidor propio.

API o servicio	Tipo	Uso
Health Services (`MeasureClient`, `PassiveMonitoringClient`)	API de plataforma/Google para Wear OS	Consultar capacidades y obtener eventos de frecuencia cardiaca con los permisos correspondientes.
Wearable Data Layer (`DataClient`, `MessageClient`)	API de Google Play services	Intercambiar registros y acciones entre reloj y teléfono.
Room / SQLite	Biblioteca Android y base de datos local	Persistir registros y observar cambios localmente.
Notificaciones Android	API del sistema Android	Crear avisos locales y dirigir al detalle/formulario correcto.
Sensores de cámara	No se usan	No existe una necesidad de cámara dentro del alcance definido.
Servidor o API web propia	No se contempla en la primera versión	No habrá cuentas, nube ni transmisión a servicios externos propios.

8. Versiones y buenas prácticas

El código fuente se alojará en el repositorio privado de GitHub `EmanuelC1601/PulsoCheck`. Se trabajará con commits pequeños y descriptivos, revisando `git status` y `git diff` antes de registrar cada avance. La asesora deberá recibir acceso como colaboradora para validar el progreso; el permiso se considera concedido solo después de que la invitación de GitHub sea aceptada.

No se incluirán en Git `local.properties`, claves de firma, contraseñas, archivos de compilación ni datos reales de salud. Los cambios se explicarán antes de integrarlos. El código nuevo mantendrá nombres descriptivos y comentarios útiles para explicar decisiones, evitando comentarios que solo repitan la instrucción de una línea.

9. Conclusiones

La propuesta se desarrollará con dos módulos Android, una arquitectura por capas y MVVM en presentación. Repository, flujos observables, Adapter y estados explícitos ayudarán a separar interfaz, reglas, almacenamiento, sensor y comunicación. Health Services obtendrá las lecturas disponibles; Room conservará los datos localmente; Wearable Data Layer los transferirá sin depender de una conexión permanente.

La primera versión no usará cámara, nube, diagnóstico ni recomendaciones de tratamiento. Las notificaciones dependerán de capacidades y permisos reales del dispositivo, y las lecturas atrasadas se identificarán como históricas. El desarrollo y la validación se realizarán por hitos, con datos simulados identificados y pruebas finales en el reloj y teléfono disponibles. Estas decisiones mantienen el alcance explicable, reducen permisos innecesarios y permiten revisar el progreso mediante Git.

10. Relación con la propuesta inicial

Este complemento amplía el documento inicial de PulsoCheck. Los requisitos de usuario, sensores, funcionamiento sin conexión, regla de episodios, historial, almacenamiento y validación se mantienen como fuente de alcance. Las decisiones aquí descritas se actualizarán cuando se implementen y validen en los dispositivos.